Lab 2 – Smart City Software Requirements Specification

Carlos Moll

Old Dominion University

CS411W, Spring 2026

Dr. Sumaya Sanober

March 14, 2026

Version 1

Table of Contents

Table of Contents 2
List of Figures 3
List of Tables 3
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 References 6
1.5 Overview 7
2. Overall Description 8
2.1 Product Perspective 8
2.2 Product Functions 8
2.3 User Characteristics 10
2.4 Constraints 10
2.5 Assumptions and Dependencies 11

List of Figures

Figure 1 – Smart City Prototype Major Functional Component Diagram 14

List of Tables

Table 1 – Smart City Feature Description and Prototype Implementation 15

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for the Smart City mobile application prototype. It is written for the development team and is meant to serve as a reference for the implementation of the project. This document is not intended for general audiences.

The SRS covers the system's expected behavior, architecture, features, constraints, and dependencies. It follows the IEEE 830-1998 standard for Software Requirements Specifications.

1.2 Scope

Smart City is a mobile application built to address the growing challenges of urbanization and overcrowding in cities. The app gives residents and commuters access to a navigation system, a community forum, and real-time alerts so they can get around and stay informed. In the future, city planners and officials will also be able to use the platform's crowdsourced data to help with urban planning decisions.

The Smart City prototype will provide the following core capabilities:

- User account creation, authentication, and access management.
- A community forum allowing residents to post, respond to, and view local updates.
- A navigation system leveraging third-party mapping APIs (Mapbox and OpenStreetMap) to support real-time route planning and traffic awareness.
- Real-time alerts and notifications for traffic, community events, and account-related activity (simulated in prototype).
- A relational database for storing user-generated data and application state.
- Data anonymity to ensure user privacy.

The Smart City prototype will not include the following functionality:

- A long-term data platform or web application for city officials (deferred to future versions).
- Machine learning, predictive analytics, or data visualization features (deferred to future versions).
- A dynamic queue for the navigation system (dropped due to scope and time constraints).
- A parking location feature for businesses (deferred to future consideration).

1.3 Definitions, Acronyms, and Abbreviations

Access Control: Controls that enforce authorized access to information and system resources in line with policy.

Android: An open-source mobile operating system used on smartphones and tablets.

Anomaly Detection: Identifying data patterns that deviate from expected behavior.

Application Programming Interface (API): A set of features and rules that enable software to interact with other software, acting as a contract between programs.

Backend: The server-side portion of an application responsible for data storage, business logic, and processing, not directly exposed to users.

Bun: A fast all-in-one JavaScript runtime, bundler, and test runner used as the development toolchain for Smart City.

Citizen Feedback Loop: A two-way process where institutions act on and report back about citizen input to sustain engagement and trust.

Crowdsourced Reporting: Gathering information by soliciting contributions from a large group of people, typically via the internet.

ElysiaJS: A high-performance web framework for building backend server applications with Bun.

Expo: An open-source framework for building cross-platform mobile applications using React Native.

Frontend: The client-side portion of an application exposed to users for direct interaction.

Internet of Things (IoT): Systems that combine sensing, computing, communication, and actuation to connect physical and cyber objects.

iOS: A closed-source mobile operating system used in Apple iPhone and iPad devices.

JavaScript: A high-level programming language used for both frontend and backend development in Smart City.

Mapbox: A third-party mapping platform providing geospatial data, tile maps, and navigation APIs.

MySQL: An open-source relational database management system used for structured data storage in Smart City.

MySQL Workbench: A graphical tool for managing and designing MySQL databases.

Node.js: A server-side JavaScript runtime environment used for backend development.

Open Data (Open Data Access): Data that anyone can freely use, re-use, and redistribute, with at most attribution or share-alike requirements.

OpenStreetMap: A collaborative, open-source geographic database used to provide map and navigation data.

Predictive AI / AI-Powered Forecasting: Using data, statistical algorithms, and machine learning to estimate the likelihood of future outcomes.

Privacy-by-Design: Building systems so that, by design and by default, only the personal data necessary for a purpose are processed.

Prototype: A working version of Smart City demonstrating core features within the project scope and time constraints.

React Native: A framework for building native mobile applications using JavaScript, used in Smart City via Expo.

Real-World Product (RWP): The fully-featured version of Smart City as originally specified, prior to prototype scope adjustments.

Software Requirements Specification (SRS): A document describing the requirements, features, and constraints of a software system for the development team.

Sprint: A time-boxed development cycle used in agile software development methodology.

Virtual Machine (VM): A digital version of a physical computer used to host the Smart City database server.

1.4 References

[1] J. Gorzelany, "Here's where commuters lose the most time and money sitting in traffic, study shows," Forbes, Aug. 30, 2024. [Online]. Available: https://www.forbes.com/sites/jimgorzelany/2024/08/30/heres-where-commuters-lose-the-most-time-and-money-sitting-in-traffic-study-shows/

[2] H. Joyce, "Urban overcrowding: A major driver of pollution crises," ShunWaste, Mar. 26, 2025. [Online]. Available: https://shunwaste.com/article/how-is-overcrowding-in-cities-causing-more-pollution

[3] G. King, "Data sharing: The next step in municipal collaboration," Data-Command, n.d. [Online]. Available: https://www.data-command.com/the-cloud/data-sharing-the-next-step-in-municipal-collaboration/

[4] S. Lee, "Understanding urban overcrowding: The social and economic implications of overcrowding in cities," Number Analytics, May 27, 2025. [Online]. Available: https://www.numberanalytics.com/blog/ultimate-guide-to-overcrowding-in-urban-sociology

[5] K. Luoma, "How does population growth impact rapid urbanization?" Population Education, Nov. 21, 2016. [Online]. Available: https://populationeducation.org/how-does-population-growth-impact-rapid-urbanization/

[6] C. Moll, "Lab 1 – Smart City Product Description," Old Dominion University, CS411W, Norfolk, VA, Feb. 28, 2026.

[7] D. Poindexter, "The effects of overcrowding and the behavioral sink theory," The Borgen Project, Jan. 24, 2018. [Online]. Available: https://borgenproject.org/theory-behavioral-sink/

[8] H. Smith, "Post-COVID L.A. traffic analysis: Has rush hour changed?" Los Angeles Times, Jun. 18, 2021. [Online]. Available: https://www.latimes.com/california/story/2021-06-18/post-covid-l-a-traffic-analysis-has-rush-hour-changed

[9] N. Weinberg, "How can cities be preemptive and effective in preventing overcrowding?" Data-Smart City Solutions, Aug. 11, 2016. [Online]. Available:

https://datasmart.hks.harvard.edu/news/article/how-can-cities-be-preemptive-and-effective-in-preventing-overcrowding-888

[10] Zurich Insurance Group, "The risks of rapid urbanization in developing countries," Apr. 13, 2023. [Online]. Available: https://www.zurich.com/knowledge/topics/global-risks/the-risks-of-rapid-urbanization-in-developing-countries

1.5 Overview

The rest of this document covers the overall description of the Smart City prototype. Section 2.1 describes the product perspective and system architecture. Section 2.2 breaks down each feature and its implementation status. Sections 2.3, 2.4, and 2.5 cover the user characteristics, constraints, and assumptions for the prototype.

2. Overall Description

2.1 Product Perspective

Smart City is a new standalone mobile application that works alongside a set of third-party services. The system is built on a three-tier client-server architecture with a presentation layer, an application layer, and a data layer.

The presentation layer is the mobile frontend, built with Expo (a React Native framework), and runs on both iOS and Android. The application layer is the backend server, built with ElysiaJS on the Bun runtime, which handles business logic, API routing, and communication with the database. The data layer is a MySQL database hosted on a virtual machine and stores all user accounts, forum posts, and navigation data.

The navigation system relies on two external APIs: Mapbox and OpenStreetMap for mapping and routing data. These connections are shown in the diagram below.

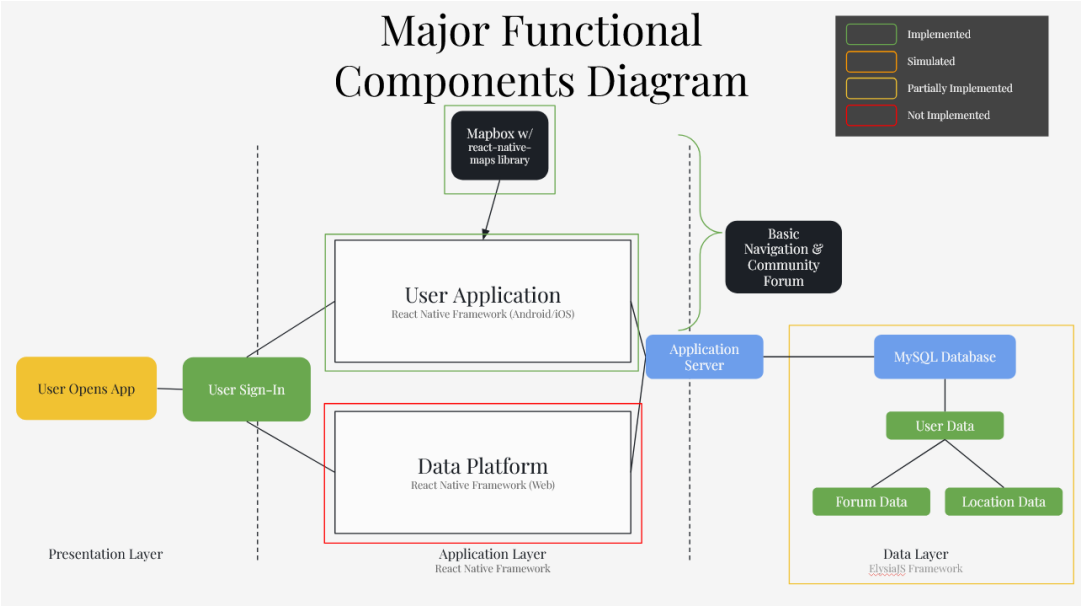

Figure 1 – Smart City Prototype Major Functional Component Diagram

2.2 Product Functions

Smart City's features are grouped into three categories: Account Management, User Application, and Data Platform. Table 1 shows each feature, how it was planned in the real-world product (RWP), its current implementation status in the prototype, and the reason for any changes.

Table 1

Smart City Feature Description and Prototype Implementation

Category	Feature	RWP	Prototype	Reasoning
Account Management	Login / Authentication	Fully Implemented	Fully Implemented	Essential for prototype
	Account Creation / Deletion	Fully Implemented	Fully Implemented	Essential for prototype
	Manage Account Access / Permissions	Fully Implemented	Fully Implemented	Essential for prototype
User Application	Access Community Forum	Fully Implemented	Fully Implemented	Essential for prototype
	Navigation (Dynamic Queue)	Fully Implemented	Partially Implemented	Dynamic Queue removed due to complexity; basic navigation still implemented
	Alerts and Notifications	Fully Implemented	Partially Implemented	Simulated feature until API route handling is resolved
	Add Parking Locations	Fully Implemented	Not Implemented	Dropped due to time constraints; may be revisited
Data Platform	Central Information Hub (Database)	Fully Implemented	Partially Implemented	Database implemented; long-term web platform dropped due to time constraints
	Data Analytics and Insights	Fully Implemented	Not Implemented	Dropped; dependent on long-term data platform
	Visualize Data	Fully Implemented	Not Implemented	Dropped; dependent on long-term data platform
	Data Anonymity	Fully Implemented	Fully Implemented	Essential for prototype

Features marked as Fully Implemented will be completed as specified in the original product design. Features marked as Partially Implemented will retain core functionality but with reduced scope or simulated data. Features marked as Not Implemented have been deferred from the prototype due to time and complexity constraints and may be revisited in future iterations.

2.3 User Characteristics

Smart City is designed to support three user groups, each with distinct roles, technical expectations, and usage patterns.

Residents and Commuters

Residents and commuters are the main users of the prototype. They use the app to get navigation help, read forum posts, and receive alerts. No technical knowledge is needed beyond basic smartphone use. For the case study, this group mainly includes ODU students and residents from nearby neighborhoods like Colonial Place and Park Place.

Community Contributors

Community contributors are users who actively post to the community forum and share local updates, traffic reports, or safety information. They interact with the same mobile interface as residents. Minimal technical skill is required beyond basic smartphone use. These users serve as the primary source of crowdsourced data for the platform.

City Officials / City Planners (Future User Group)

City planners and officials are a future user group that will use a data platform to view trends and insights pulled from resident data. In the prototype, this role is simulated through the same shared login as other users. City planners will need more data literacy than average residents since they will use analytics dashboards for planning purposes. This role is not yet fully implemented.

2.4 Constraints

The following constraints apply to the Smart City prototype:

- **Third-Party API Dependency:** The navigation system depends on the availability and rate limits of the Mapbox and OpenStreetMap APIs. Downtime or changes to these services could affect the functionality of the navigation feature.
- **Development Timeline:** The prototype must be delivered within a single academic semester. This constraint directly limits the scope of features that can be fully implemented.
- **Team Technical Proficiency:** Several team members are learning tools such as Bun, ElysiaJS, and MySQL for the first time during active development, which may introduce delays.
- **Platform Requirements:** Users must have a smartphone running iOS or Android with access to the iOS App Store or Google Play Store, and an active internet connection.

- **Hosting Environment:** The database is hosted on a virtual machine within the ODU infrastructure, which constrains scalability and availability outside of the university environment.

2.5 Assumptions and Dependencies

The following assumptions and external dependencies apply to the Smart City prototype:

- **User Device:** It is assumed that all users have access to a smartphone (iOS or Android) with a working internet connection and the ability to download applications from a mobile app marketplace.
- **Internet Connectivity:** The application requires continuous internet connection to access the community forum, navigation features, and alert notifications. Offline functionality is not supported in the prototype.
- **Mapbox API:** The navigation system depends on Mapbox's mapping and routing services. It is assumed that API credentials remain valid and that usage remains within free-tier or allocated rate limits.
- **OpenStreetMap:** Supplemental geospatial data is sourced from OpenStreetMap, which is assumed to remain publicly available and up to date.
- **Bun and ElysiaJS Stability:** The backend is built on Bun and ElysiaJS. It is assumed that these frameworks remain stable and compatible with each other throughout the development lifecycle.
- **ODU Virtual Machine:** The database is hosted on an ODU-provided virtual machine. It is assumed that this environment remains accessible and operational during prototype development and testing.
- **MySQL Compatibility:** It is assumed that the MySQL instance deployed on the virtual machine is compatible with the schema and query patterns used by the application.